

Composite

1. Explicación del patrón

Composite es un patrón estructural que permite representar composiciones de objetos con profundidad variable mediante una estructura en forma de árbol.

La idea principal es tratar de la misma manera a los objetos simples y a los objetos compuestos.

Un objeto simple, llamado hoja, no contiene otros objetos.

Un objeto compuesto, llamado compuesto o contenedor, puede contener hojas y también otros objetos compuestos.

Gracias a esto, el cliente puede trabajar con cualquier elemento de la estructura usando una interfaz común, sin tener que saber si está tratando con un objeto individual o con una composición formada por muchos objetos.

El patrón encapsula la composición respecto al cliente. Por eso, el cliente no necesita conocer la profundidad del árbol ni la estructura interna de los objetos compuestos.

2. Cuándo se utiliza

Se utiliza el patrón Composite cuando:

- Es necesario representar jerarquías de composición en un sistema.
- El modelo del problema puede organizarse como un árbol.
- Hay objetos simples y objetos compuestos que deben tratarse de forma uniforme.
- Los clientes no deben saber si se comunican con una hoja o con un compuesto.
- Se quiere ocultar al cliente la profundidad o complejidad de la composición.
- Un objeto compuesto puede contener otros objetos que, a su vez, también pueden ser compuestos.
- Se quiere aplicar una operación de forma recursiva sobre una estructura completa.
- Se quiere evitar procesar de forma diferente los elementos simples y los elementos compuestos.
- Se quiere simplificar el código cliente usando una interfaz común para todos los elementos.

3. Cómo aplicarlo

Para aplicar Composite se suelen definir estos participantes:

Componente

Es la clase abstracta o interfaz común para todos los objetos de la composición.

Define las operaciones que podrán usar los clientes tanto sobre hojas como sobre compuestos.

Responsabilidades:

- Declarar la interfaz común.

- Definir operaciones compartidas por hojas y compuestos.
- Declarar, si procede, métodos para añadir o eliminar componentes.

Operaciones habituales:

- `solicita()`
- `agregaComponente(componente)`
- `suprimeComponente(componente)`

En el ejemplo de empresas del material, el componente sería:

- `Empresa`

Hoja

Es una clase concreta que representa un objeto simple dentro del árbol.

Una hoja no contiene otros componentes.

Responsabilidades:

- Implementar las operaciones comunes definidas por `Componente`.
- Realizar el trabajo real cuando recibe una petición.
- No gestionar hijos, porque no tiene componentes internos.

En el ejemplo de empresas del material, la hoja sería:

- `EmpresaSinFilial`

Compuesto

Es una clase concreta que representa un objeto formado por otros componentes.

Puede contener hojas y otros compuestos.

Responsabilidades:

- Guardar una colección de componentes hijos.
- Implementar las operaciones comunes.
- Añadir componentes.
- Eliminar componentes.
- Recorrer sus hijos cuando recibe una petición.
- Delegar la operación en sus componentes internos.

En el ejemplo de empresas del material, el compuesto sería:

- `EmpresaMadre`

Cliente

Es la clase que usa los objetos de la composición.

Responsabilidades:

- Trabajar con los objetos mediante la interfaz `Componente` .
- No distinguir entre hojas y compuestos.
- Enviar peticiones sin conocer la estructura interna del árbol.

4. Forma práctica de aplicarlo

Pasos:

1. Identificar si el problema puede representarse como una estructura en árbol.
2. Separar los elementos simples de los elementos compuestos.
3. Crear una interfaz o clase abstracta común llamada `Componente` .
4. Definir en `Componente` las operaciones que el cliente necesita usar.
5. Crear una clase `Hoja` para los elementos simples.
6. Crear una clase `Compuesto` para los elementos que pueden contener otros elementos.
7. Añadir al compuesto una colección de objetos de tipo `Componente` .
8. Implementar en el compuesto métodos para agregar y eliminar componentes.
9. En las operaciones principales del compuesto, recorrer sus hijos y delegar la petición.
10. Hacer que el cliente trabaje siempre con el tipo `Componente` .
11. Evitar que el cliente tenga que comprobar si un objeto es hoja o compuesto.

5. Idea clave

El cliente no debe preocuparse de si está usando un objeto simple o un grupo de objetos.

Todos los elementos comparten una misma interfaz.

Así, una operación puede aplicarse a una hoja o a un árbol completo de objetos de la misma forma.

6. Funcionamiento interno

Cuando el cliente envía una petición a un componente, pueden ocurrir dos cosas:

Si el componente es una hoja

La hoja procesa la petición directamente.

No delega en otros objetos porque no tiene hijos.

Si el componente es un compuesto

El compuesto puede hacer un procesamiento previo.

Después, normalmente recorre sus componentes hijos y les envía la misma petición.

Cada hijo puede ser una hoja o puede ser otro compuesto.

Si un hijo también es compuesto, repetirá el mismo proceso.

Finalmente, el compuesto puede hacer un procesamiento posterior.

Este comportamiento permite recorrer toda la estructura de forma recursiva.

7. Ejemplos del material

Ejemplo de empresas y filiales

En el material aparece un sistema de venta de vehículos donde se quieren representar empresas cliente.

El objetivo es conocer el número de vehículos que poseen y ofrecer mantenimiento para su parque de vehículos.

El problema aparece porque algunas empresas tienen filiales y otras no.

Además, una filial también puede tener sus propias filiales, por lo que la estructura puede crecer en varios niveles.

Una solución directa sería tratar de forma distinta:

- Empresas sin filiales.
- Empresas con filiales.

Pero eso haría que la aplicación dependiera demasiado de la composición interna de cada empresa.

Composite resuelve el problema unificando ambos tipos de empresa mediante una estructura recursiva.

Elementos del ejemplo:

- `Empresa` : componente.
- `EmpresaSinFilial` : hoja.
- `EmpresaMadre` : compuesto.
- `filiales` : colección de componentes dentro de `EmpresaMadre` .

La clase `Empresa` contiene la interfaz destinada a los clientes.

Métodos indicados en el material:

- `agregaVehiculo()`
- `calculaCosteMantenimiento()`
- `agregaFilial()`

`EmpresaSinFilial` calcula su coste de mantenimiento directamente.

`EmpresaMadre` calcula su coste teniendo en cuenta sus propios datos y los de sus filiales.

Para ello, recorre sus filiales y llama a `calculaCosteMantenimiento()` en cada una.

Ejemplo de estructura de empresas

El material muestra una estructura como esta:

- `empresaA` : `EmpresaMadre`

- `empresaB: EmpresaSinFilial`
- `empresaC: EmpresaMadre`
 - `empresaD: EmpresaSinFilial`

El cliente puede pedir el cálculo de mantenimiento a `empresaA` .

No necesita saber que dentro hay otra empresa madre ni que esa empresa contiene más filiales.

La llamada se propaga recursivamente por el árbol.

Ejemplo de elementos educativos

El material también muestra una estructura con elementos educativos.

Aparece una clase general:

- `ElementoEducativo`

Con atributos y operaciones como:

- `elementos:int`
- `agregaElemento()`
- `getNombreElemento()`

Y clases concretas como:

- `DepartamentoDocencia`
- `DepartamentoInvestigacion`
- `Universidad`

La idea es representar una jerarquía donde algunos elementos pueden contener otros y todos pueden tratarse mediante una interfaz común.

8. Ejemplo conceptual de uso

Sin Composite, el cliente tendría que hacer comprobaciones como:

`si es empresa sin filial, calcular directamente`

`si es empresa madre, recorrer sus filiales`

Con Composite, el cliente solo hace:

`empresa.calculaCosteMantenimiento()`

Da igual si `empresa` es una hoja o un compuesto.

Cada objeto sabe cómo responder.

9. Ventajas prácticas

- Permite representar estructuras jerárquicas en forma de árbol.
- Permite tratar objetos simples y compuestos de la misma manera.

- Oculta al cliente la profundidad de la composición.
- Reduce el acoplamiento entre el cliente y la estructura interna.
- Facilita añadir nuevos tipos de componentes.
- Permite aplicar operaciones de forma recursiva.
- Simplifica el código cliente.
- Evita condicionales para distinguir entre hojas y compuestos.

10. Cuidado

Composite puede hacer que la interfaz común sea demasiado general.

Algunas operaciones, como añadir o eliminar hijos, tienen sentido para los compuestos, pero no para las hojas.

Por ejemplo, una `EmpresaMadre` puede tener filiales, pero una `EmpresaSinFilial` no debería tenerlas.

Hay dos formas habituales de tratar este problema:

- Declarar las operaciones de composición en el componente común, aunque algunas hojas las implementen vacías o lancen error.
- Declarar las operaciones de composición solo en los compuestos, haciendo la interfaz más segura pero menos uniforme.

La elección depende de si se prioriza la uniformidad para el cliente o la claridad de responsabilidades.

11. Diferencia práctica con otros patrones estructurales

Composite sirve para representar jerarquías parte-todo mediante árboles.

Adapter sirve para hacer compatibles interfaces distintas.

Decorator sirve para añadir responsabilidades a un objeto manteniendo una estructura parecida.

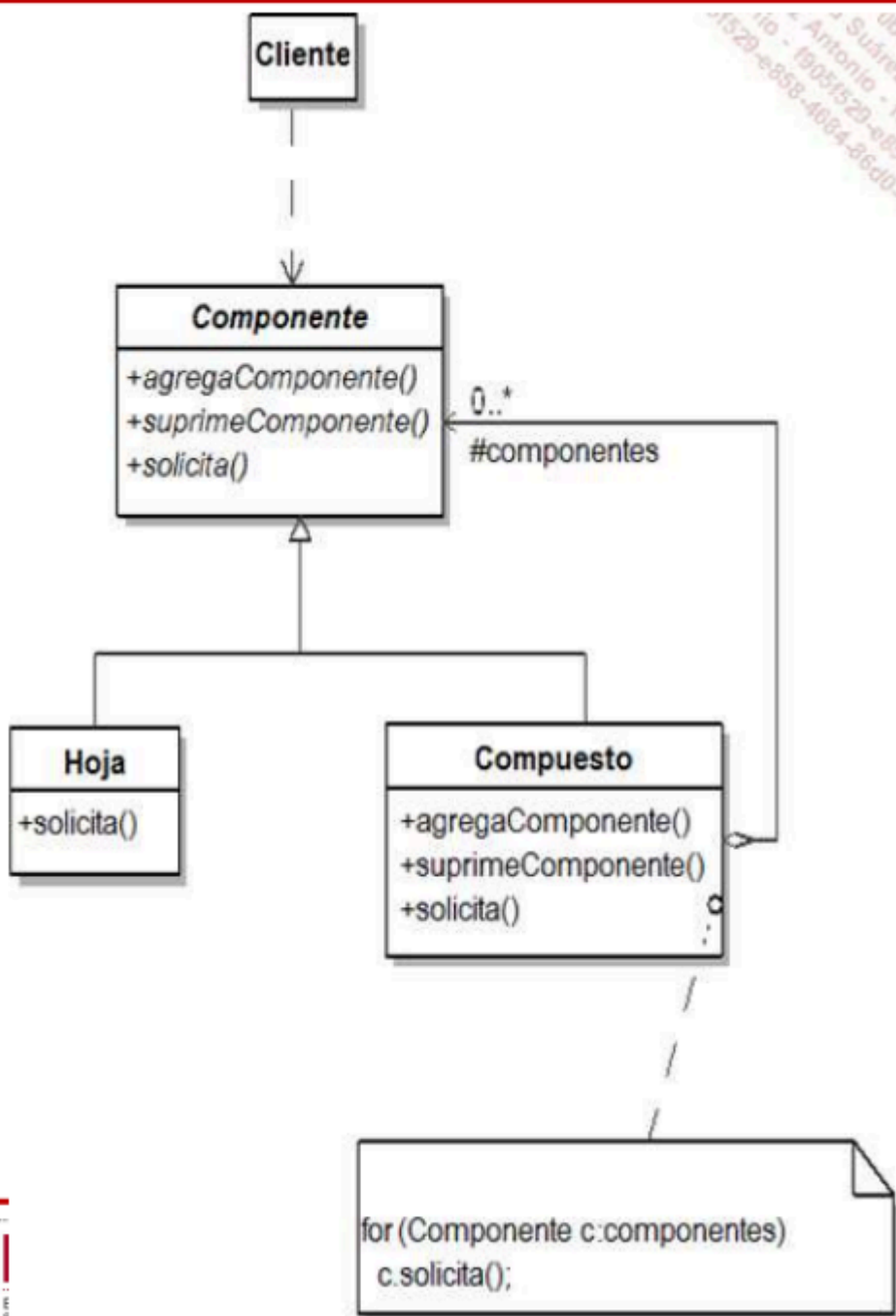
Facade sirve para simplificar el acceso a un subsistema.

Proxy sirve para controlar el acceso a un objeto.

Por tanto:

- Composite se centra en componer objetos.
- Adapter se centra en adaptar interfaces.
- Decorator se centra en añadir comportamiento.
- Facade se centra en simplificar.
- Proxy se centra en controlar el acceso.

12. Plantilla



13. Ejemplo

